```
{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# LOGOS prototype 5 : does the guardian know what it does not know? (NASA C-MAPSS)\n",
    "\n",
    "Question : the warning model learns from one kind of engine fault (FD001). When it meets engines with a fault type it never saw (FD003), it stays silent on many of them. Can a separate \"do I recognise this?\" check tell us, **live and before failure**, which engines the warning model cannot be trusted on?\n",
    "\n",
    "- The guardian is trained only on FD001 engines 1-80. FD003 contains a fault type FD001 does not have (per NASA 's documentation).\n",
    "- Runtime : Colab, CPU. Each analysis cell takes 1 to 3 minutes.\n",
    "- Honest limits are at the end."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Step 1 : download the data"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "import urllib.request, zipfile, os\n",
    "import numpy as np, pandas as pd\n",
    "\n",
    "url = \"https://kr0k0tsch.de/rul-datasets/CMAPSSData.zip\"\n",
    "if not os.path.exists(\"cmapss\"):\n",
    "    urllib.request.urlretrieve(url, \"cmapss.zip\")\n",
    "    with zipfile.ZipFile(\"cmapss.zip\") as z:\n",
    "        z.extractall(\"cmapss\")\n",
    "print(\"OK:\", sorted(os.listdir(\"cmapss\"))[:5])"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Step 2 : what happens outside the guardian 's experience (silent misses) and a first familiarity check"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "import glob\n",
    "from sklearn.linear_model import LogisticRegression\n",
    "from sklearn.preprocessing import StandardScaler\n",
    "from sklearn.decomposition import PCA\n",
    "from sklearn.metrics import roc_auc_score\n",
    "\n",
    "H, W, L = 30, 20, 60\n",
    "cols = [\"unit\", \"cycle\"] + [\"set%d\" % i for i in range(1, 4)] + [\"s%d\" %
```

```
i for i in range(1, 22)]\n",
    "sensors = [\"s2\", \"s3\", \"s4\", \"s7\", \"s8\", \"s9\", \"s11\", \"s12\",
\"s13\", \"s14\", \"s15\", \"s17\", \"s20\", \"s21\"]\n",
    "d = len(sensors)\n",
    "\n",
    "def window_features(X):\n",
    "    feats, idx = [], []\n",
    "    tt = np.arange(W) - (W - 1) / 2\n",
    "    for t in range(W, len(X) + 1):\n",
    "        w = X[t - W:t]; m = w.mean(0)\n",
    "        feats.append(np.r_[m, w[-1], (tt[:, None] * (w - m)).sum(0) / (tt **
2).sum(), w.std(0)]); idx.append(t - 1)\n",
    "    return np.array(feats), np.array(idx)\n",
    "\n",
    "def load(name, units=None):\n",
    "    data = pd.read_csv(glob.glob(\"cmapss/**/\" + name, recursive=True)[0],
sep=r\"\\s+\", header=None, names=cols)\n",
    "    F, y, ep, tp, ft = [], [], [], [], {}\n",
    "    for u in (units if units is not None else
sorted(data[\"unit\"].unique())):\n",
    "        g = data[data[\"unit\"] == u]; f, idx =
window_features(g[sensors].values); t_f = len(g) - 1\n",
    "        F.append(f); ep.append(np.full(len(idx), u)); tp.append(idx); ft[u] =
t_f\n",
    "        y.append((((t_f - idx) <= H) & ((t_f - idx) > 0)).astype(int))\n",
    "    F, y, ep, tp = np.vstack(F), np.concatenate(y), np.concatenate(ep),
np.concatenate(tp)\n",
    "    tf = np.array([ft[u] for u in ep], dtype=float)\n",
    "    P = dict(ep=ep, t=tp, tf=tf, healthy=~(tp >= tf - L), win=(tp < tf) & (tp >=
tf - L), n_fail=len(ft), per1000=(~(tp >= tf - L)).sum() / 1000.0)\n",
    "    return F, y, P\n",
    "\n",
    "def evaluate_alarm(score, P, thr, gap=10):\n",
    "    alarm = score > thr\n",
    "    m = alarm & P[\"healthy\"]; e_, t_ = P[\"ep\"][m], P[\"t\"][m]\n",
    "    events = 1 + int(((np.diff(e_) != 0) | (np.diff(t_) > gap)).sum()) if len(t_)
else 0\n",
    "    mw = alarm & P[\"win\"]\n",
    "    warned = len(np.unique(P[\"ep\"][mw])) if mw.any() else 0\n",
    "    return warned / P[\"n_fail\"], events / P[\"per1000\"]\n",
    "\n",
    "# --- the guardian knows ONLY engines 1-80 of FD001 (one kind of fault) ---\n",
    "Ftr, ytr, Ptr = load(\"train_FD001.txt\", list(range(1, 81)))\n",
    "Fk, yk, Pk = load(\"train_FD001.txt\", list(range(81, 101)))     # known kind,
unseen engines\n",
    "Fu, yu, Pu = load(\"train_FD003.txt\")                            # FD003:
includes a fault type FD001 never had\n",
    "\n",
    "sc = StandardScaler().fit(Ftr)\n",
    "rng = np.random.default_rng(0)\n",
    "models = []\n",
    "for k in range(5):                                              # 5 models
trained on resampled engines\n",
    "    ids = rng.choice(len(Ftr), len(Ftr), replace=True)\n",
    "    models.append(LogisticRegression(max_iter=3000).fit(sc.transform(Ftr[ids]),
ytr[ids]))\n",
    "risk = lambda F_: np.mean([m.predict_proba(sc.transform(F_))[:, 1] for m in
models], axis=0)\n",
    "\n",
    "lo, hi = 0.0, 1.0\n",
    "s_tr = risk(Ftr)\n",
    "for _ in range(30):                                             # alarm level: 1
false alarm per 1000 healthy cycles on training engines\n",
    "    mid = (lo + hi) / 2\n",
    "    if evaluate_alarm(s_tr, Ptr, mid)[1] > 1: lo = mid\n",
    "    else: hi = mid\n",
```

```
    "thr = hi\n",
    "\n",
    "print(\"1) Same alarm level, applied to engines the guardian knows vs engines
with an unseen fault type\")\n",
    "for nm, F_, P_ in ((\"known kind (FD001, unseen engines)\", Fk, Pk), (\"partly
unknown (FD003)\", Fu, Pu)):\n",
    "    w_, fa = evaluate_alarm(risk(F_), P_, thr)\n",
    "    print(\"  %-38s warned %3.0f%% of engines  | false alarms %.1f per 1000
healthy cycles\" % (nm, 100 * w_, fa))\n",
    "\n",
    "# \"Do I recognise this?\" : how far is a situation from everything seen in
training (PCA distance)\n",
    "pca = PCA(n_components=10).fit(sc.transform(Ftr))\n",
    "var = pca.explained_variance_\n",
    "fam = lambda F_: (pca.transform(sc.transform(F_)) ** 2 / var).sum(1)\n",
    "f_tr, f_k, f_u = fam(Ftr), fam(Fk), fam(Fu)\n",
    "cut = np.percentile(f_tr, 99)\n",
    "print(\"\\n2) 'Do I recognise this?' check (distance from what the guardian was
trained on)\")\n",
    "print(\"  share of situations flagged as unfamiliar:  known kind %.0f%%  |  FD003
%.0f%%\" % (100 * np.mean(f_k > cut), 100 * np.mean(f_u > cut)))\n",
    "print(\"  AUROC for telling FD003 from known engines: %.3f  (0.5 = cannot tell,
1.0 = perfect)\" % roc_auc_score(np.r_[np.zeros(len(f_k)), np.ones(len(f_u))],
np.r_[f_k, f_u]))\n",
    "print(\"\\nPlain-language rule for the report: if a situation is flagged as
unfamiliar, LOGOS says\")\n",
    "print(\"  'I do not recognise this situation. My warning score is unreliable
here. A human must check.'\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Reading (our run): on engines of the known kind the warning model warned 100%; on
FD003 only 56%, with fewer false alarms: it stayed silent rather than noisy. A simple
row-level familiarity check separated FD003 from known engines only weakly (AUROC
0.668)."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Step 3: engine by engine, three familiarity checks"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "import glob\n",
    "from sklearn.linear_model import LogisticRegression\n",
    "from sklearn.preprocessing import StandardScaler\n",
    "from sklearn.decomposition import PCA\n",
    "from sklearn.metrics import roc_auc_score\n",
    "\n",
    "H, W, L = 30, 20, 60\n",
    "cols = [\"unit\", \"cycle\"] + [\"set%d\" % i for i in range(1, 4)] + [\"s%d\" %
i for i in range(1, 22)]\n",
    "sensors = [\"s2\", \"s3\", \"s4\", \"s7\", \"s8\", \"s9\", \"s11\", \"s12\",
\"s13\", \"s14\", \"s15\", \"s17\", \"s20\", \"s21\"]\n",
    "d = len(sensors)\n",
    "\n",
```

```
    "def window_features(X):\n",
    "    feats, idx = [], []\n",
    "    tt = np.arange(W) - (W - 1) / 2\n",
    "    for t in range(W, len(X) + 1):\n",
    "        w = X[t - W:t]; m = w.mean(0)\n",
    "        feats.append(np.r_[m, w[-1], (tt[:, None] * (w - m)).sum(0) / (tt **
2).sum(), w.std(0)]); idx.append(t - 1)\n",
    "    return np.array(feats), np.array(idx)\n",
    "\n",
    "def load(name, units=None):\n",
    "    data = pd.read_csv(glob.glob(\"cmapss/**/\" + name, recursive=True)[0],
sep=r\"\\s+\", header=None, names=cols)\n",
    "    F, y, ep, tp, ft = [], [], [], [], {}\n",
    "    for u in (units if units is not None else
sorted(data[\"unit\"].unique())):\n",
    "        g = data[data[\"unit\"] == u]; f, idx =
window_features(g[sensors].values); t_f = len(g) - 1\n",
    "        F.append(f); ep.append(np.full(len(idx), u)); tp.append(idx); ft[u] =
t_f\n",
    "        y.append((((t_f - idx) <= H) & ((t_f - idx) > 0)).astype(int))\n",
    "    F, y, ep, tp = np.vstack(F), np.concatenate(y), np.concatenate(ep),
np.concatenate(tp)\n",
    "    tf = np.array([ft[u] for u in ep], dtype=float)\n",
    "    P = dict(ep=ep, t=tp, tf=tf, healthy=~(tp >= tf - L), win=(tp < tf) & (tp >=
tf - L), n_fail=len(ft), per1000=(~(tp >= tf - L)).sum() / 1000.0)\n",
    "    return F, y, P\n",
    "\n",
    "def evaluate_alarm(score, P, thr, gap=10):\n",
    "    alarm = score > thr\n",
    "    m = alarm & P[\"healthy\"]; e_, t_ = P[\"ep\"][m], P[\"t\"][m]\n",
    "    events = 1 + int(((np.diff(e_) != 0) | (np.diff(t_) > gap)).sum()) if len(t_)
else 0\n",
    "    mw = alarm & P[\"win\"]\n",
    "    warned = len(np.unique(P[\"ep\"][mw])) if mw.any() else 0\n",
    "    return warned / P[\"n_fail\"], events / P[\"per1000\"]\n",
    "\n",
    "# --- the guardian knows ONLY engines 1-80 of FD001 (one kind of fault) ---\n",
    "Ftr, ytr, Ptr = load(\"train_FD001.txt\", list(range(1, 81)))\n",
    "Fk, yk, Pk = load(\"train_FD001.txt\", list(range(81, 101)))     # known kind,
unseen engines\n",
    "Fu, yu, Pu = load(\"train_FD003.txt\")                            # FD003:
includes a fault type FD001 never had\n",
    "\n",
    "sc = StandardScaler().fit(Ftr)\n",
    "rng = np.random.default_rng(0)\n",
    "models = []\n",
    "for k in range(5):                                              # 5 models
trained on resampled engines\n",
    "    ids = rng.choice(len(Ftr), len(Ftr), replace=True)\n",
    "    models.append(LogisticRegression(max_iter=3000).fit(sc.transform(Ftr[ids]),
ytr[ids]))\n",
    "risk = lambda F_: np.mean([m.predict_proba(sc.transform(F_))[:, 1] for m in
models], axis=0)\n",
    "\n",
    "lo, hi = 0.0, 1.0\n",
    "s_tr = risk(Ftr)\n",
    "for _ in range(30):                                             # alarm level: 1
false alarm per 1000 healthy cycles on training engines\n",
    "    mid = (lo + hi) / 2\n",
    "    if evaluate_alarm(s_tr, Ptr, mid)[1] > 1: lo = mid\n",
    "    else: hi = mid\n",
    "thr = hi\n",
    "\n",
    "# three ways to ask \"do I recognise this situation?\"\n",
    "from sklearn.neighbors import NearestNeighbors\n",
    "Ztr = sc.transform(Ftr)\n",
```

```
    "pca = PCA(n_components=10).fit(Ztr); var = pca.explained_variance_\n",
    "sub = Ztr[np.random.default_rng(1).choice(len(Ztr), min(8000, len(Ztr)),
replace=False)]\n",
    "nn = NearestNeighbors(n_neighbors=10).fit(sub)\n",
    "fam = {\n",
    "    \"A: PCA distance\": lambda F_: (pca.transform(sc.transform(F_)) ** 2 /
var).sum(1),\n",
    "    \"B: nearest-neighbour distance\": lambda F_:
nn.kneighbors(sc.transform(F_))[0].mean(1),\n",
    "    \"C: largest single deviation\": lambda F_:
np.abs(sc.transform(F_)).max(1),\n",
    "}\n",
    "cuts = {k: np.percentile(f(Ftr), 99) for k, f in fam.items()}\n",
    "\n",
    "# engine by engine: which FD003 engines did the guardian MISS, and did a
familiarity check warn about them?\n",
    "auc = lambda yy, ss: roc_auc_score(yy, ss) if len(set(yy)) > 1 else 0.5\n",
    "alarm = risk(Fu) > thr\n",
    "eng = np.unique(Pu[\"ep\"])\n",
    "missed = np.array([not (alarm & Pu[\"win\"] & (Pu[\"ep\"] == e)).any() for e in
eng])\n",
    "print(\"FD003 engines: %d  | warned %d  | MISSED %d\" % (len(eng), (~missed).sum(),
missed.sum()))\n",
    "print(\"\\nCan a familiarity check tell us which engines will be missed? (AUROC:
0.5 = no, 1.0 = perfectly)\")\n",
    "shares = {}\n",
    "for k, f in fam.items():\n",
    "    flag = f(Fu) > cuts[k]\n",
    "    shares[k] = np.array([flag[Pu[\"ep\"] == e].mean() for e in eng])\n",
    "    print(\"  %-32s AUROC = %.3f\" % (k, auc(missed, shares[k])))\n",
    "\n",
    "best = max(shares, key=lambda k: auc(missed, shares[k]))\n",
    "sh = shares[best]; cut_e = np.median(sh)\n",
    "fl = sh > cut_e\n",
    "print(\"\\nUsing the best check (%s): engines with more unfamiliar situations
than the median are 'flagged'\" % best)\n",
    "print(\"  flagged engines   : %d, of which MISSED by the warning model: %.0f%%\"
% (fl.sum(), 100 * missed[fl].mean() if fl.any() else 0))\n",
    "print(\"  unflagged engines : %d, of which MISSED by the warning model: %.0f%%\"
% ((~fl).sum(), 100 * missed[~fl].mean() if (~fl).any() else 0))\n",
    "print(\"\\nIf flagged engines are missed far more often, LOGOS can say: 'do not
trust my silence on this engine, a human must check.'\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Reading (our run): counted over the whole life of each engine, all three checks
separated the engines the warning model missed from the ones it caught almost
perfectly (AUROC about 1.0). Caution: this uses the whole life of the engine,
including the moments close to failure, and the cut at the median happened to suit a
roughly 50/50 mix."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Step 4: the live version (only data up to now; rule fixed before seeing
FD003)"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
```

```
   "execution_count": null,
   "outputs": [],
   "source": [
    "import glob\n",
    "from sklearn.linear_model import LogisticRegression\n",
    "from sklearn.preprocessing import StandardScaler\n",
    "from sklearn.decomposition import PCA\n",
    "from sklearn.metrics import roc_auc_score\n",
    "\n",
    "H, W, L = 30, 20, 60\n",
    "cols = [\"unit\", \"cycle\"] + [\"set%d\" % i for i in range(1, 4)] + [\"s%d\" %
i for i in range(1, 22)]\n",
    "sensors = [\"s2\", \"s3\", \"s4\", \"s7\", \"s8\", \"s9\", \"s11\", \"s12\",
\"s13\", \"s14\", \"s15\", \"s17\", \"s20\", \"s21\"]\n",
    "d = len(sensors)\n",
    "\n",
    "def window_features(X):\n",
    "    feats, idx = [], []\n",
    "    tt = np.arange(W) - (W - 1) / 2\n",
    "    for t in range(W, len(X) + 1):\n",
    "        w = X[t - W:t]; m = w.mean(0)\n",
    "        feats.append(np.r_[m, w[-1], (tt[:, None] * (w - m)).sum(0) / (tt **
2).sum(), w.std(0)]); idx.append(t - 1)\n",
    "    return np.array(feats), np.array(idx)\n",
    "\n",
    "def load(name, units=None):\n",
    "    data = pd.read_csv(glob.glob(\"cmapss/**/\" + name, recursive=True)[0],
sep=r\"\\s+\", header=None, names=cols)\n",
    "    F, y, ep, tp, ft = [], [], [], [], {}\n",
    "    for u in (units if units is not None else
sorted(data[\"unit\"].unique())):\n",
    "        g = data[data[\"unit\"] == u]; f, idx =
window_features(g[sensors].values); t_f = len(g) - 1\n",
    "        F.append(f); ep.append(np.full(len(idx), u)); tp.append(idx); ft[u] =
t_f\n",
    "        y.append((((t_f - idx) <= H) & ((t_f - idx) > 0)).astype(int))\n",
    "    F, y, ep, tp = np.vstack(F), np.concatenate(y), np.concatenate(ep),
np.concatenate(tp)\n",
    "    tf = np.array([ft[u] for u in ep], dtype=float)\n",
    "    P = dict(ep=ep, t=tp, tf=tf, healthy=~(tp >= tf - L), win=(tp < tf) & (tp >=
tf - L), n_fail=len(ft), per1000=(~(tp >= tf - L)).sum() / 1000.0)\n",
    "    return F, y, P\n",
    "\n",
    "def evaluate_alarm(score, P, thr, gap=10):\n",
    "    alarm = score > thr\n",
    "    m = alarm & P[\"healthy\"]; e_, t_ = P[\"ep\"][m], P[\"t\"][m]\n",
    "    events = 1 + int(((np.diff(e_) != 0) | (np.diff(t_) > gap)).sum()) if len(t_)
else 0\n",
    "    mw = alarm & P[\"win\"]\n",
    "    warned = len(np.unique(P[\"ep\"][mw])) if mw.any() else 0\n",
    "    return warned / P[\"n_fail\"], events / P[\"per1000\"]\n",
    "\n",
    "# --- the guardian knows ONLY engines 1-80 of FD001 (one kind of fault) ---\n",
    "Ftr, ytr, Ptr = load(\"train_FD001.txt\", list(range(1, 81)))\n",
    "Fk, yk, Pk = load(\"train_FD001.txt\", list(range(81, 101)))     # known kind,
unseen engines\n",
    "Fu, yu, Pu = load(\"train_FD003.txt\")                            # FD003:
includes a fault type FD001 never had\n",
    "\n",
    "sc = StandardScaler().fit(Ftr)\n",
    "rng = np.random.default_rng(0)\n",
    "models = []\n",
    "for k in range(5):                                              # 5 models
trained on resampled engines\n",
    "    ids = rng.choice(len(Ftr), len(Ftr), replace=True)\n",
    "    models.append(LogisticRegression(max_iter=3000).fit(sc.transform(Ftr[ids]),
```

```
ytr[ids]))\n",
    "risk = lambda F_: np.mean([m.predict_proba(sc.transform(F_))[:, 1] for m in
models], axis=0)\n",
    "\n",
    "lo, hi = 0.0, 1.0\n",
    "s_tr = risk(Ftr)\n",
    "for _ in range(30):                                              # alarm level: 1
false alarm per 1000 healthy cycles on training engines\n",
    "    mid = (lo + hi) / 2\n",
    "    if evaluate_alarm(s_tr, Ptr, mid)[1] > 1: lo = mid\n",
    "    else: hi = mid\n",
    "thr = hi\n",
    "\n",
    "# three ways to ask \"do I recognise this situation?\"\n",
    "from sklearn.neighbors import NearestNeighbors\n",
    "Ztr = sc.transform(Ftr)\n",
    "pca = PCA(n_components=10).fit(Ztr); var = pca.explained_variance_\n",
    "sub = Ztr[np.random.default_rng(1).choice(len(Ztr), min(8000, len(Ztr)),
replace=False)]\n",
    "nn = NearestNeighbors(n_neighbors=10).fit(sub)\n",
    "fam = {\n",
    "    \"A: PCA distance\": lambda F_: (pca.transform(sc.transform(F_)) ** 2 /
var).sum(1),\n",
    "    \"B: nearest-neighbour distance\": lambda F_:
nn.kneighbors(sc.transform(F_))[0].mean(1),\n",
    "    \"C: largest single deviation\": lambda F_:
np.abs(sc.transform(F_)).max(1),\n",
    "}\n",
    "cuts = {k: np.percentile(f(Ftr), 99) for k, f in fam.items()}\n",
    "\n",
    "# ---- LIVE version: only data up to the current cycle is used, and the rule is
fixed BEFORE seeing FD003 ----\n",
    "K = 30                                                            # look at the
last 30 cycles\n",
    "fB = fam[\"B: nearest-neighbour distance\"]\n",
    "ind_k, ind_u = fB(Fk) > cuts[\"B: nearest-neighbour distance\"], fB(Fu) >
cuts[\"B: nearest-neighbour distance\"]\n",
    "\n",
    "def first_flag(ind, P, e, s):\n",
    "    m = P[\"ep\"] == e\n",
    "    i, t = ind[m].astype(float), P[\"t\"][m]\n",
    "    roll = np.convolve(i, np.ones(K) / K, mode=\"full\")[:len(i)]\n",
    "    ok = np.where((roll >= s) & (np.arange(len(i)) >= K - 1))[0]\n",
    "    return t[ok[0]] if len(ok) else None\n",
    "\n",
    "# choose the rule on known engines FD001 81-100 (never used for training): at
most 5% of them may ever be flagged\n",
    "eng_k = np.unique(Pk[\"ep\"])\n",
    "rule = 1.0\n",
    "for s_ in np.arange(0.2, 1.01, 0.05):\n",
    "    if np.mean([first_flag(ind_k, Pk, e, s_) is not None for e in eng_k]) <=
0.05:\n",
    "        rule = s_; break\n",
    "print(\"rule chosen on known engines: flag an engine when >= %.0f%% of its last
%d cycles look unfamiliar\" % (100 * rule, K))\n",
    "print(\"known engines ever flagged: %.0f%% (this is the false-flag rate)\" % (100
* np.mean([first_flag(ind_k, Pk, e, rule) is not None for e in eng_k])))\n",
    "\n",
    "eng = np.unique(Pu[\"ep\"])\n",
    "alarm = risk(Fu) > thr\n",
    "missed = np.array([not (alarm & Pu[\"win\"] & (Pu[\"ep\"] == e)).any() for e in
eng])\n",
    "tfail = np.array([Pu[\"tf\"][Pu[\"ep\"] == e][0] for e in eng])\n",
    "tflag = [first_flag(ind_u, Pu, e, rule) for e in eng]\n",
    "lead = np.array([(tf_ - t_) if t_ is not None else np.nan for tf_, t_ in
zip(tfail, tflag)])\n",
```

```
    "flagged = ~np.isnan(lead)\n",
    "\n",
    "print(\"\\nFD003: %d engines | warned by the model %d | MISSED %d\" % (len(eng),
(~missed).sum(), missed.sum()))\n",
    "print(\"Engines the model MISSED:\")\n",
    "print(\"  flagged 'unfamiliar' before the failure : %.0f%%\" % (100 *
flagged[missed].mean() if missed.any() else 0))\n",
    "print(\"  flagged at least 60 cycles before failure: %.0f%%\" % (100 *
np.mean(lead[missed] >= 60) if missed.any() else 0))\n",
    "if missed.any() and flagged[missed].any():\n",
    "    print(\"  median time of the flag: %.0f cycles before failure\" %
np.nanmedian(lead[missed]))\n",
    "print(\"Engines the model WARNED about, flagged anyway: %.0f%%\" % (100 *
flagged[~missed].mean() if (~missed).any() else 0))\n",
    "print(\"\\nShare of MISSED engines already flagged by cycle:\")\n",
    "for c in (60, 90, 120, 150):\n",
    "    got = [(t_ is not None and t_ <= c) for t_ in np.array(tflag,
dtype=object)[missed]]\n",
    "    print(\"  cycle %3d: %.0f%%\" % (c, 100 * np.mean(got) if len(got) else
0))\n",
    "print(\"\\nIf most missed engines are flagged well before they fail, LOGOS can
say it early:\")\n",
    "print(\"  'I do not recognise how this engine is behaving. My silence is not a
good sign. A human must check.'\")"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Reading (our run): the rule was chosen on 20 known engines never used for
training (5% flagged). On FD003 every one of the 44 engines the warning model missed
was flagged as unfamiliar before failure, 98% of them at least 60 cycles early (median
182 cycles). 11% of the engines the model did warn about were flagged as well.\n",
    "\n",
    "Caution found afterwards: in Steps 3 and 4 the distance cut was computed on the
training rows, which are also the reference points of the nearest-neighbour check, so
the cut was biased. Step 5 fixes this and repeats the test."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Step 5: repeated, corrected test on four dataset pairs (3 random splits each)"
   ]
  },
  {
   "cell_type": "code",
   "metadata": {},
   "execution_count": null,
   "outputs": [],
   "source": [
    "import glob, time\n",
    "from sklearn.linear_model import LogisticRegression\n",
    "from sklearn.preprocessing import StandardScaler\n",
    "from sklearn.neighbors import NearestNeighbors\n",
    "\n",
    "H, W, L, K = 30, 20, 60, 30\n",
    "cols = [\"unit\", \"cycle\"] + [\"set%d\" % i for i in range(1, 4)] + [\"s%d\" %
i for i in range(1, 22)]\n",
    "sensors = [\"s2\", \"s3\", \"s4\", \"s7\", \"s8\", \"s9\", \"s11\", \"s12\",
\"s13\", \"s14\", \"s15\", \"s17\", \"s20\", \"s21\"]\n",
    "\n",
    "def window_features(X):\n",
    "    feats, idx = [], []\n",
```

```
    "    tt = np.arange(W) - (W - 1) / 2\n",
    "    for t in range(W, len(X) + 1):\n",
    "        w = X[t - W:t]; m = w.mean(0)\n",
    "        feats.append(np.r_[m, w[-1], (tt[:, None] * (w - m)).sum(0) / (tt **
2).sum(), w.std(0)]); idx.append(t - 1)\n",
    "    return np.array(feats), np.array(idx)\n",
    "\n",
    "D = {}                                                  # all four files,
features computed once\n",
    "t0 = time.time()\n",
    "for name in (\"FD001\", \"FD002\", \"FD003\", \"FD004\"):\n",
    "    data = pd.read_csv(glob.glob(\"cmapss/**/train_%s.txt\" % name,
recursive=True)[0], sep=r\"\\s+\", header=None, names=cols)\n",
    "    F, y, ep, tp, tf = [], [], [], [], []\n",
    "    for u, g in data.groupby(\"unit\"):\n",
    "        f, idx = window_features(g[sensors].values); t_f = len(g) - 1\n",
    "        F.append(f); ep.append(np.full(len(idx), u)); tp.append(idx);
tf.append(np.full(len(idx), float(t_f)))\n",
    "        y.append((((t_f - idx) <= H) & ((t_f - idx) > 0)).astype(int))\n",
    "    D[name] = dict(F=np.vstack(F), y=np.concatenate(y), ep=np.concatenate(ep),
t=np.concatenate(tp), tf=np.concatenate(tf))\n",
    "    print(name, \"ready (%d engines)\" % len(np.unique(D[name][\"ep\"])), \"|
% .0fs\" % (time.time() - t0))\n",
    "\n",
    "def part(name, units):\n",
    "    d_ = D[name]; m = np.isin(d_[\"ep\"], units)\n",
    "    P = dict(ep=d_[\"ep\"][m], t=d_[\"t\"][m], tf=d_[\"tf\"][m])\n",
    "    P[\"healthy\"] = ~(P[\"t\"] >= P[\"tf\"] - L); P[\"win\"] = (P[\"t\"] <
P[\"tf\"]) & (P[\"t\"] >= P[\"tf\"] - L)\n",
    "    P[\"n_fail\"] = len(units); P[\"per1000\"] = P[\"healthy\"].sum() /
1000.0\n",
    "    return d_[\"F\"][m], d_[\"y\"][m], P\n",
    "\n",
    "def evaluate_alarm(score, P, thr, gap=10):\n",
    "    alarm = score > thr\n",
    "    m = alarm & P[\"healthy\"]; e_, t_ = P[\"ep\"][m], P[\"t\"][m]\n",
    "    events = 1 + int(((np.diff(e_) != 0) | (np.diff(t_) > gap)).sum()) if len(t_)
else 0\n",
    "    return events / P[\"per1000\"]\n",
    "\n",
    "def first_flag(ind, P, e, s):\n",
    "    m = P[\"ep\"] == e\n",
    "    i, t = ind[m].astype(float), P[\"t\"][m]\n",
    "    roll = np.convolve(i, np.ones(K) / K, mode=\"full\")[:len(i)]\n",
    "    ok = np.where((roll >= s) & (np.arange(len(i)) >= K - 1))[0]\n",
    "    return t[ok[0]] if len(ok) else None\n",
    "\n",
    "def run_pair(known, new, seed):\n",
    "    rng = np.random.default_rng(seed)\n",
    "    units = np.array(sorted(np.unique(D[known][\"ep\"]))); rng.shuffle(units)\n",
    "    a, b = int(0.6 * len(units)), int(0.8 * len(units))\n",
    "    tr_u, ca_u, cb_u = units[:a], units[a:b], units[b:]          # train  | set A
(distance cut) | set B (flag rule)\n",
    "    Ftr, ytr, Ptr = part(known, tr_u); Fca, yca, Pca = part(known, ca_u); Fcb,
ycb, Pcb = part(known, cb_u)\n",
    "    Fab, yab, Pab = part(known, np.r_[ca_u, cb_u])\n",
    "    Fn, yn, Pn = part(new, np.unique(D[new][\"ep\"]))\n",
    "    sc = StandardScaler().fit(Ftr); Ztr = sc.transform(Ftr)\n",
    "    models = []\n",
    "    for k in range(5):\n",
    "        ids = rng.choice(len(Ftr), len(Ftr), replace=True)\n",
    "        models.append(LogisticRegression(max_iter=3000).fit(Ztr[ids],
ytr[ids]))\n",
    "    risk = lambda F_: np.mean([m.predict_proba(sc.transform(F_))[:, 1] for m in
models], axis=0)\n",
    "    lo, hi = 0.0, 1.0; s_ab = risk(Fab)                     # warning level set
```

```
on engines never used for training\n",
    "    for _ in range(30):\n",
    "        mid = (lo + hi) / 2\n",
    "        if evaluate_alarm(s_ab, Pab, mid) > 1: lo = mid\n",
    "        else: hi = mid\n",
    "    thr = hi\n",
    "    sub = Ztr[rng.choice(len(Ztr), min(8000, len(Ztr)), replace=False)]\n",
    "    nn = NearestNeighbors(n_neighbors=10).fit(sub)\n",
    "    dist = lambda F_: nn.kneighbors(sc.transform(F_))[0].mean(1)\n",
    "    cut = np.percentile(dist(Fca), 99)                    # distance cut from
UNSEEN known engines (not training rows)\n",
    "    ind_cb, ind_n = dist(Fcb) > cut, dist(Fn) > cut\n",
    "    ecb = np.unique(Pcb[\"ep\"]); rule = 1.0\n",
    "    for s_ in np.arange(0.2, 1.01, 0.05):\n",
    "        if np.mean([first_flag(ind_cb, Pcb, e, s_) is not None for e in ecb]) <=
0.05:\n",
    "            rule = s_; break\n",
    "    eng = np.unique(Pn[\"ep\"]); alarm = risk(Fn) > thr\n",
    "    missed = np.array([not (alarm & Pn[\"win\"] & (Pn[\"ep\"] == e)).any() for e
in eng])\n",
    "    tfail = np.array([Pn[\"tf\"][Pn[\"ep\"] == e][0] for e in eng])\n",
    "    tflag = [first_flag(ind_n, Pn, e, rule) for e in eng]\n",
    "    lead = np.array([(x - y_) if y_ is not None else np.nan for x, y_ in
zip(tfail, tflag)])\n",
    "    fl = ~np.isnan(lead)\n",
    "    return dict(n=len(eng), missed=int(missed.sum()), rule=rule,\n",
    "                m_flag=100 * fl[missed].mean() if missed.any() else np.nan,\n",
    "                m_early=100 * np.mean(lead[missed] >= 60) if missed.any() else
np.nan,\n",
    "                w_flag=100 * fl[~missed].mean() if (~missed).any() else
np.nan,\n",
    "                known_flag=100 * np.mean([first_flag(ind_cb, Pcb, e, rule) is not
None for e in ecb]))\n",
    "\n",
    "def show(v): return \"n/a\" if v != v else \"%.0f%%\" % v\n",
    "print(\"Fixed version: the distance cut now comes from known engines that were
NOT used for training.\")\n",
    "print(\"%-14s %-10s %-7s %-22s %-14s %s\" % (\"learns -> meets\", \"missed\",
\"rule\", \"missed ones flagged\", \">=60 cyc early\", \"warned-but-flagged | known
flagged\"))\n",
    "for kn, nw in [(\"FD001\", \"FD003\"), (\"FD003\", \"FD001\"), (\"FD002\",
\"FD004\"), (\"FD004\", \"FD002\")]:\n",
    "    R = [run_pair(kn, nw, s) for s in range(3)]\n",
    "    m = lambda k: np.nan if np.all(np.isnan([r[k] for r in R])) else
np.nanmean([r[k] for r in R])\n",
    "    lo_, hi_ = (np.nanmin([r[\"m_flag\"] for r in R]), np.nanmax([r[\"m_flag\"]
for r in R])) if R[0][\"missed\"] else (np.nan, np.nan)\n",
    "    print(\"%-14s %-10s %-7s %-22s %-14s %s  | %s\" % (kn + \" -> \" + nw, \"%.0f
of %d\" % (np.mean([r[\"missed\"] for r in R]), R[0][\"n\"]),\n",
    "          \"%.0f%%\" % (100 * np.mean([r[\"rule\"] for r in R])), \"%s (%s to
%s)\" % (show(m(\"m_flag\")), show(lo_), show(hi_)),\n",
    "          show(m(\"m_early\")), show(m(\"w_flag\")), show(m(\"known_flag\"))))"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "Reading (our run, corrected version):\n",
    "\n",
    "| learns -> meets | engines missed | missed ones flagged | flagged >=60 cycles
early | warned engines flagged | known engines flagged |\n",
    "|---|---|---|---|---|---|\n",
    "| FD001 -> FD003 | 44 of 100 | 100% (100 to 100) | 99% | 3% | 5% |\n",
    "| FD003 -> FD001 | 0 of 100 | n/a | n/a | 3% | 5% |\n",
    "| FD002 -> FD004 | 101 of 249 | 8% (5 to 12) | 4% | 2% | 3% |\n",
```

```
    "| FD004 -> FD002 | 0 of 260 | n/a | n/a | 4% | 3% |\n",
    "\n",
    "The familiarity check works on the single-condition pair (FD001 -> FD003) and fails on the six-condition pair (FD002 -> FD004), where the warning model misses 40% of engines and the check catches almost none. Our hypothesis, not tested: raw sensor values there are dominated by changing operating conditions, so a distance check cannot see the unseen fault. Normalising sensors per operating condition is the usual remedy and is the next step."
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Limits (please read)\n",
    "\n",
    "- NASA C-MAPSS is a simulation of jet engines, not cars and not real fleet logs.\n",
    "- Four pairs were tested (3 splits each). The check worked on one (FD001 to FD003) and failed on another (FD002 to FD004); two pairs had no missed engines to find.\n",
    "- The flag appears very early (median 182 cycles before failure). That suggests the check may be noticing that these engines look different from the start, not necessarily the unseen fault itself. NASA does not label the fault type per engine, so we could not verify this.\n",
    "- The rule is calibrated on a small set of known engines (20 to 50), so the false-flag rate is uncertain.\n",
    "- Detecting unfamiliar inputs and predicting failure from sensors are both established research areas. The point here is the combination: a system that says \"I do not recognise this, my silence is not reassuring, a human must check\".\n",
    "- No safety assessment of any kind."
   ]
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "Python 3",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
   "name": "python"
  },
  "colab": {
   "provenance": []
  }
 },
 "nbformat": 4,
 "nbformat_minor": 4
}
```